Retrieval-Augmented Schema Grounding for LLM-Based Text-to-SQL with a Polyglot PostgreSQL and Vector Database Architecture

Gowthamkrishna R
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India
Reg. No. 23MIA1087

Aakash R
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India
Reg. No. 23MIA1011

***Abstract*—Large language models (LLMs) can translate a plain-English question into SQL, but the usual approach pastes the whole database schema into every prompt. This raises token cost and gives the model tables it does not need. We present a Text-to-SQL system that grounds the model with retrieval from a vector database. Each table is written as a short plain-English card, embedded with gemini-embedding-001, and stored in ChromaDB next to 52 solved example questions. At query time the system retrieves the most similar tables, adds their foreign-key neighbours, retrieves similar solved examples, and sends only this context to Gemini. Generated SQL passes a read-only validator before it runs on PostgreSQL. We ran a controlled ablation on a 30-question project suite, a 32-question Spider subset, and 40 past user queries. Schema annotations (table descriptions and allowed column values) raised execution accuracy from 86.7% to 96.7%. Table retrieval cut prompt size by 49% (1,108 to 561 tokens) on the project schema and by 20% on Spider, with 100% table recall; on the small project schema it cost two questions (96.7% to 90.0%, not significant at n = 30). Example retrieval raised Spider accuracy from 62.5% to 71.9% and agreement with earlier correct answers from 85.0% to 97.5%. The validator blocked 25 of 25 attack queries with no false blocks on 222 valid queries. The results show where retrieval helps, where it does not, and what it costs.**

***Index Terms*—Text-to-SQL, large language models, retrieval-augmented generation, vector database, schema linking, ChromaDB, PostgreSQL, SQL injection**

I. Introduction

Most organisational data sits in relational databases, and getting answers out of them still needs SQL and knowledge of the schema. Analysts, managers and students who lack either depend on a data team for simple lookups. Text-to-SQL systems remove this step by turning a natural-language question into an executable query. Recent LLMs do this well on public benchmarks [1]–[4], which makes a practical application realistic.

Building that application raises problems that benchmark papers rarely cover. The common prompt pastes every table into the context. As schemas grow, this costs more tokens per query and gives the model unrelated tables that look similar to the right ones. Generated SQL can also be unsafe: prompt injection can steer an LLM-backed application into destructive statements [5], and poisoned models can emit hidden injections [6]. Finally, many systems report one accuracy number without showing which part of the pipeline produced it.

We address these points with a retrieval-augmented generation (RAG) design [7] built on a polyglot data layer. PostgreSQL stores the data users query. ChromaDB, a vector database, stores embeddings of table descriptions and of solved example questions. For each question the system retrieves only the relevant tables and examples, builds a compact prompt, and validates the SQL before execution. A dashboard shows the generated SQL, the result and the retrieved context, and lets the user switch retrieval on or off.

The contributions of this work are:

- An end-to-end Text-to-SQL system with a polyglot PostgreSQL and ChromaDB data layer, a read-only SQL validator and an interactive dashboard, packaged with Docker.
- A schema-retrieval method that embeds plain-English table cards, selects tables by cosine similarity and expands the selection along foreign keys, together with example retrieval that excludes the test question to prevent leakage.
- Schema annotation with table descriptions and allowed values of low-cardinality columns, which gave the largest single accuracy gain in our study.
- A controlled ablation over five prompt configurations on three evaluation sets, reporting execution accuracy with confidence intervals, table recall, prompt tokens, latency and validator block rates.

Section II reviews related work. Section III defines the task. Section IV describes the architecture and database design, and Section V the retrieval method. Section VI gives the experimental setup, Section VII the results, Section VIII the limitations, and Section IX concludes.

II. Related Work

A. Benchmarks

Spider [1] is the standard cross-domain benchmark, with 10,181 questions over 200 databases. Systems are scored by exact match or by execution accuracy, which compares the result rows of the predicted and gold queries. BIRD [2] adds 12,751 questions over 95 larger, noisier databases. It shows that understanding database values is harder than parsing alone: GPT-4 reached about 55% execution accuracy against about 93% for humans.

B. LLM Prompting Methods

Rajkumar et al. [8] showed that LLMs can produce SQL from a schema-in-prompt without fine-tuning. DIN-SQL [3] splits the task into schema linking, classification, generation and self-correction, reaching 85.3% on the Spider test set at the cost of several LLM calls per query. C3 [9] reaches about 82% zero-shot with calibrated prompts. DAIL-SQL [4] compares prompt designs systematically and finds that the choice of few-shot examples matters most; it selects examples by question and query similarity. MAC-SQL [10] uses separate selector, decomposer and refiner agents, which helps with large schemas but multiplies calls.

C. Schema Linking and Retrieval

Schema linking decides which tables and columns a question needs. RESDSQL [11] trains a cross-encoder to rank and filter schema items before decoding. CHESS [12] retrieves relevant values and schema items from large BIRD databases before generation. These methods build on dense retrieval [13], [14], where text is mapped to vectors and compared by similarity, and on approximate nearest-neighbour indexes such as HNSW [15]. Retrieval-augmented generation [7] applies the same idea to supply an LLM with external context at inference time.

D. Applied Systems, Surveys and Security

DB-GPT [16] packages LLM-database interaction with local models for privacy. Mirror [17] combines querying, summarisation and visualisation in one interface. FinSQL [18] adapts Text-to-SQL to financial data. Two recent surveys [19], [20] list deployment, robustness and cost as open problems. On security, Pedro et al. [5] show that prompt-to-SQL injection works against LLM-integrated web applications, and ToxicSQL [6] shows that poisoning 0.44% of fine-tuning data gives a 79.41% attack success rate. Table I summarises the works closest to ours.

TABLE I
COMPARISON OF RELATED WORK

Work	Approach	Evaluation	Gap for our setting
Spider [1]	Cross-domain benchmark	200 DBs	Clean schemas; no deployment
BIRD [2]	Value-grounded benchmark	95 DBs	Large human gap remains
DIN-SQL [3]	Decomposed prompting	Spider, BIRD	Many LLM calls per query
DAIL-SQL [4]	Example selection	Spider, BIRD	Full schema in prompt
MAC-SQL [10]	Multi-agent pipeline	BIRD, Spider	High cost; no safety layer
CHESS [12]	Retrieval for linking	BIRD	Not a user-facing system
DB-GPT [16]	Local-LLM DB agent	Case studies	No SQL validation study
Mirror [17]	Query + visualise	Custom tasks	No injection defence
P2SQL [5]	Injection attacks	7 LLMs	Attack-focused
This work	Table + example RAG, validator	Project, Spider, history	Small-schema study

E. Research Gap

Prior work either optimises benchmark accuracy with costly multi-call pipelines or builds applications without measuring which component helps. Retrieval of tables and examples has been studied on large benchmark schemas, but its cost and benefit on a small production-style schema are rarely reported, and few systems combine retrieval with a measured safety layer and a transparent interface. We study these questions in one system with a controlled ablation.

III. PROBLEM FORMULATION

Let a database have schema $S = \{t_1, \ldots, t_n\}$, where each table t_i has columns, a primary key and foreign keys. Given a question q, the system must produce a SQL query $\hat{y}$ whose result on the database equals the result of a reference query y^*. A full-schema prompt conditions the LLM on (q, S). A retrieval prompt conditions it on (q, S', E'), where $S' \subset S$ is a retrieved subset of tables and E' is a set of retrieved solved examples. The goal is to keep execution accuracy while making S' much smaller than S, and to reject any $\hat{y}$ that is not a read-only query.

Our objectives were: (O1) answer questions over a PostgreSQL database in plain English; (O2) ground generation with vector retrieval of tables and examples; (O3) block unsafe SQL before execution; and (O4) measure the effect of each component on accuracy, prompt size and latency.

IV. System Architecture and Database Design

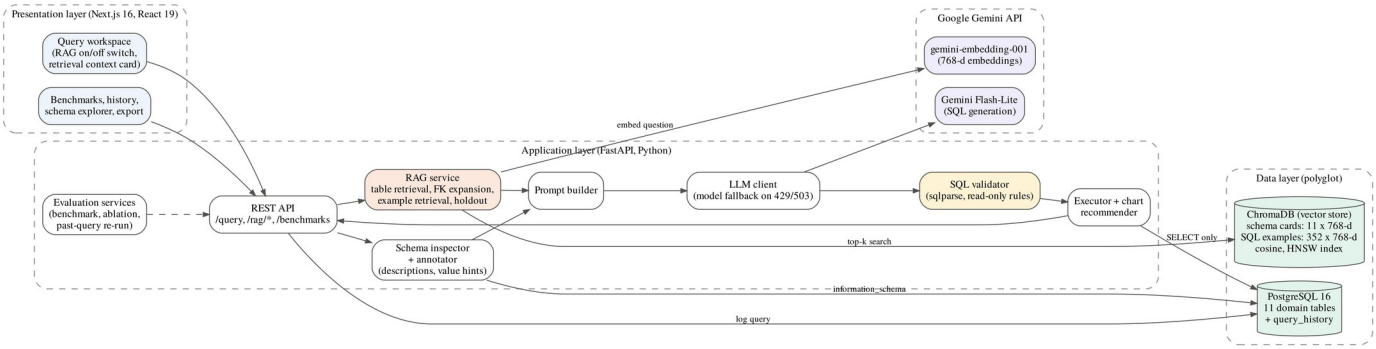


Fig. 1. System architecture. The application layer combines relational schema inspection with vector retrieval before calling the LLM; every query passes the validator before it reaches PostgreSQL.

Fig. 1 shows the three layers. The presentation layer is a Next.js dashboard with a query workspace, a RAG on/off switch, a card that shows the retrieved tables and examples, and pages for benchmarks, history and the schema. The application layer is a FastAPI service. It inspects the live schema, runs retrieval, builds the prompt, calls Gemini, validates the SQL, executes it, and recommends a chart. Evaluation services run the benchmark, the ablation and the past-query re-run from the same code path. All components run in Docker containers.

A. Relational Store

PostgreSQL 16 holds eleven tables in three domains (Fig. 2): a university domain (students, courses, enrollments, instructors, departments), an e-commerce domain (customers, orders, order items, products) and a corporate domain (employees, corporate departments). The domains share terms on purpose. Both the university and corporate domains have a department table, and both instructors and employees have a salary, which makes table selection non-trivial. A separate `query_history` table logs every prompt, query, row count, status and execution time, and later serves as an evaluation set.

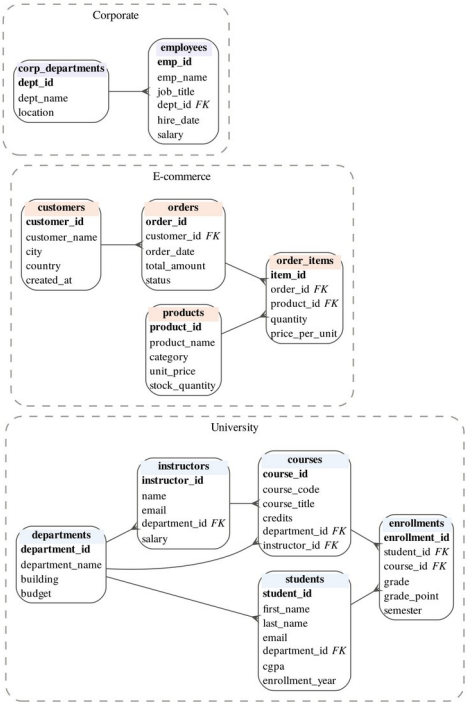


Fig. 2. Relational schema of the eleven domain tables. Crow's-foot ends mark the many side of each foreign key.

B. Vector Store

ChromaDB stores two collections. The schema collection holds one 768-dimensional vector per table card (11 vectors). The example collection holds 352 vectors: 52 solved project questions and 300 Spider training questions, each stored with its SQL and a source tag. Both collections use cosine distance with an HNSW index [15]. A signature of the card text and embedding model is stored with each collection, so the index is rebuilt only when the schema or model changes. The project runs with the example source restricted to project questions.

The two stores do different jobs. PostgreSQL answers exact, structured queries over the data. ChromaDB answers similarity queries over meaning, which a relational index cannot do. Keeping them separate lets each be indexed for its own access pattern.

V. Retrieval-Augmented Generation Method

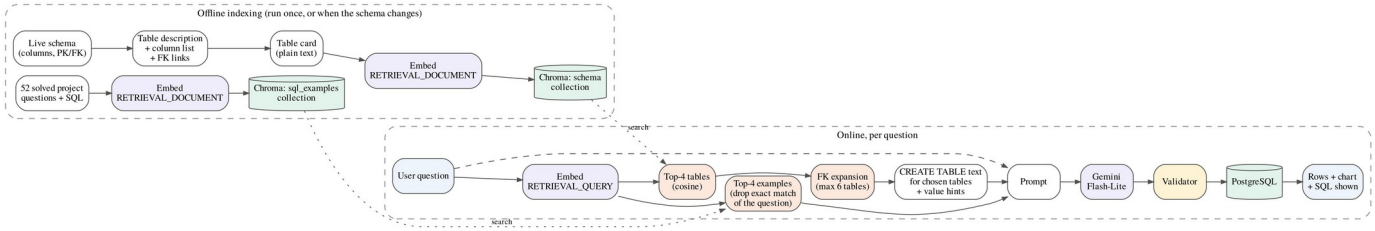


Fig. 3. RAG pipeline. Offline, table cards and solved examples are embedded and indexed. Online, the question is embedded once and used to retrieve tables and examples, which form the prompt.

A. Schema Annotation

The schema inspector reads columns and keys from `information_schema`. Two annotations are then added. First, a one-line description for each table, kept in a JSON file. Second, value hints: for text columns with at most 12 distinct values, the allowed values are listed, for example the order `status` values or the course `grade` values. The hints are cached and refreshed periodically. In the prompt, each selected table is written as a `CREATE TABLE` statement with its primary and foreign keys, preceded by its description and followed by its value hints. This text only describes existing tables; nothing is executed.

B. Table Cards and Embeddings

Each table becomes a short card: its name, its description, its column names written as words, and its foreign-key links. For example:

```
Table: courses (courses)
Description: Courses offered, with course
  code, title, credits, department and the
  instructor who teaches it
Columns: course id, course code, course
  title, credits, department id,
  instructor id
Links to: departments (via department_id),
  instructors (via instructor_id)
```

Cards are embedded with gemini-embedding-001 [21] using the document task type and an output size of 768 dimensions. Questions are embedded with the query task type. Fig. 4 shows the cosine similarity between all table cards. Tables from the same domain form clear blocks, but `departments` and `corp_departments` are the most similar pair across domains (0.90). This pair explains one of the retrieval errors in Section VII.

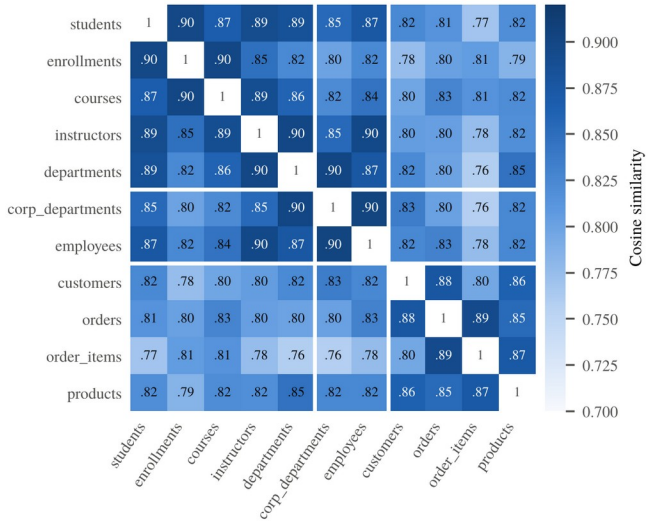


Fig. 4. Cosine similarity between the eleven table-card embeddings, read from the ChromaDB store. White lines separate the university, corporate and e-commerce domains.

C. Table Retrieval and Foreign-Key Expansion

For a question vector q and a card vector c_i, the similarity is

$$\mathrm{sim}(q, c_i) = \frac{q \cdot c_i}{\|q\| \, \|c_i\|} \tag{1}$$

The $k = 4$ tables with the highest similarity are selected. Because a query often needs tables the question does not name, the selection is expanded in two steps: tables referenced by a selected table's foreign keys are added, and then any table that references at least two selected tables is added as a bridge (for example `enrollments` between `students` and `courses`). The result is capped at six tables. Only these tables are written into the prompt. If retrieval fails for any reason, the system falls back to the full schema.

D. Example Retrieval with Holdout

The same question vector retrieves the four most similar solved examples, which are inserted into the prompt as question and SQL pairs. During evaluation, an example whose question is identical to the test question is removed. Without this holdout, a benchmark question can retrieve its own answer, and the score measures copying instead of generation.

E. Generation and Validation

The prompt combines fixed instructions, the selected schema text, the retrieved examples and the question. It is sent to Gemini Flash-Lite through its REST API; on rate-limit or overload errors the client retries and falls back to other Gemini models. The returned SQL is checked by a validator built on `sqlparse`. It strips code fences, rejects multiple statements, requires the statement to start with SELECT or WITH, and rejects 16 keywords such as DROP, DELETE, UPDATE, INSERT, ALTER, GRANT, COPY and PG_SLEEP anywhere in the token stream. Only queries that pass are executed. Fig. 5 shows the full request sequence.

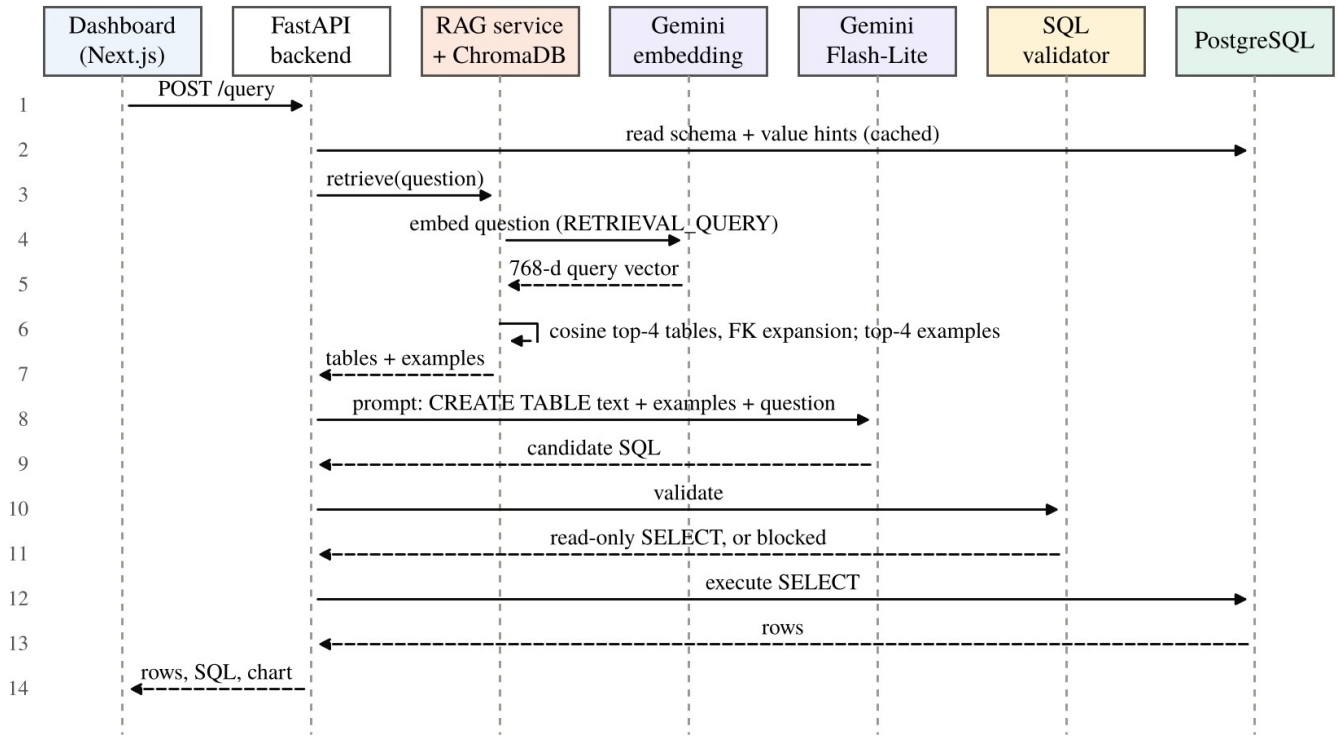


Fig. 5. Sequence of one request with retrieval enabled. Dashed arrows are responses.

VI. Experimental Setup

A. Evaluation Sets

Project suite. 30 questions over the eleven-table database with reference SQL: 10 university, 10 e-commerce, 8 corporate and 2 cross-domain questions, labelled 8 easy, 13 medium and 9 hard.

Spider subset. 40 questions from the Spider development set [1] over three databases with 6, 8 and 11 tables, sampled with a fixed seed. Eight questions failed in at least one configuration because the API returned an overload error (HTTP 503). We removed them from all configurations, leaving 32 paired questions. For Spider, examples were retrieved from the 300 Spider training questions in the example collection. Training and development questions use different databases, so they do not overlap. Expansion allowed up to eight tables.

Past queries. 42 prompts from the `query_history` log: 40 ordinary questions whose earlier SQL had run successfully, and 2 prompts that should be blocked. Each was re-run with retrieval off and on, with four parallel workers.

Validator suite. 25 attack queries covering destructive statements, stacked statements, comment tricks, time-based injection, data export and mixed case, plus 222 distinct valid queries: the reference SQL of the project suite and all SQL generated in our runs.

B. Configurations

Table II lists the configurations. B0 and D0 were run before schema annotations were added; B, C and D after. Configuration A is the original prompt of the system, which contained fixed example pairs. We found that these pairs were copies of benchmark questions and answers, so A measures recall of the answer key, not generation. We report it only to show this effect.

TABLE II
PROMPT CONFIGURATIONS

ID	Schema in prompt	Examples	Annot.
A	All tables	Fixed (leaked)	No
B0	All tables	None	No
D0	All tables	Top-4 retrieved	No
B	All tables	None	Yes
C	Top-4 + FK (max 6)	None	Yes
D	All tables	Top-4 retrieved	Yes
E	Top-4 + FK (max 6)	Top-4 retrieved	Yes

C. Metrics

Execution accuracy (EA) is the share of questions whose generated query returns the same rows as the reference query:

$$\mathrm{EA} = \frac{1}{N}\sum_{j=1}^{N} \mathbf{1}\left[R(\hat{y}_j) = R(y_j^*) \right] \tag{2}$$

where $R(y)$ is the result multiset of query y. We also report exact string match and column match (same column names). Table recall@k measures whether retrieval kept every table the reference query uses:

$$\mathrm{Recall@}k = \frac{|T(y^*) \cap S'_k|}{|T(y^*)|} \tag{3}$$

where $T(y^*)$ is the set of tables in the reference query and S'_k is the retrieved set after expansion. Prompt size is the mean prompt token count reported by the API, and latency is the mean end-to-end time per question measured by the client. For past queries we report execution success and *consistency*, the share of prompts whose new result matches the earlier successful result. For the validator we report the block rate on attacks and the false-block rate on valid queries.

With 30 questions, one question changes EA by 3.3 points, so we give 95% Wilson score intervals [22] and compare paired configurations with the exact McNemar test on the questions where they disagree [23].

D. Implementation

The backend uses Python with FastAPI and SQLAlchemy; the frontend uses Next.js, React and Recharts. Generation used Gemini Flash-Lite and embeddings used gemini-embedding-001, both through the Gemini REST API. All runs were made from a MacBook Air with the services in Docker. Every metric in this paper is read from the JSON result files saved by the evaluation scripts.

VII. Results and Discussion

TABLE III
RESULTS ON THE 30-QUESTION PROJECT SUITE

Cfg	EA % [95% CI]	Col. %	Tables	Tokens	ms
A*	100.0 [88.6, 100]	73.3	11	1,345	1,350
B0	86.7 [70.3, 94.7]	66.7	11	784	1,577
D0	90.0 [74.4, 96.5]	76.7	11	1,082	1,789
B	96.7 [83.3, 99.4]	73.3	11	1,108	1,819
C	90.0 [74.4, 96.5]	66.7	4.8	561	1,827
D	96.7 [83.3, 99.4]	80.0	11	1,406	1,932

A contains copies of benchmark answers. ms = mean end-to-end latency. B, C and D come from two runs; B scored 96.7% in both

A. Effect of Schema Annotations

Adding table descriptions and value hints raised EA from 86.7% (B0) to 96.7% (B), as Table III and Fig. 6 show. Annotations fixed three questions and broke none (McNemar p = 0.25). The fixed questions show what the annotations do. One used the stored value ‘Completed’ instead of ‘completed’, which the value hints supply. One picked the corporate department table instead of the university one, which the descriptions separate. The third added the missing join to the department name. The cost was about 324 extra tokens per prompt (784 to 1,108). This was the largest gain of any single change.

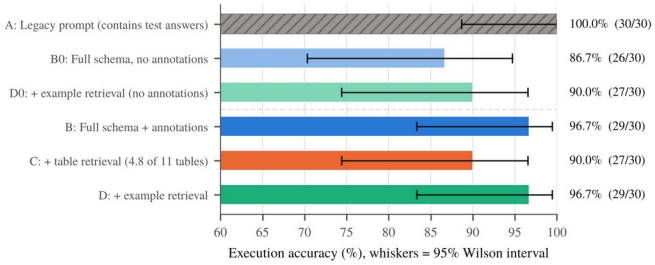


Fig. 6. Execution accuracy on the project suite with 95% Wilson intervals. The top three bars use the schema without annotations, the bottom three with annotations. A is hatched because its prompt contains benchmark answers.

B. Effect of Example Retrieval

On the project suite, example retrieval helped only before annotations: D0 scored 90.0% against 86.7% for B0 (two fixed, one broken). After annotations, D and B both scored 96.7% and failed the same single question, while D cost about 300 more tokens. The suite is too easy to separate them.

On the Spider subset, where the schemas are unfamiliar, example retrieval raised EA from 62.5% to 71.9% (Table IV). It fixed three questions and broke none (p = 0.25). On past queries it raised consistency from 85.0% to 97.5% (Fig. 7), fixing six prompts and breaking one (p = 0.125). Execution success stayed at 97.5% in both modes, so the gain is in answering the question correctly, not in avoiding errors. The off mode here used the original prompt with fixed examples.

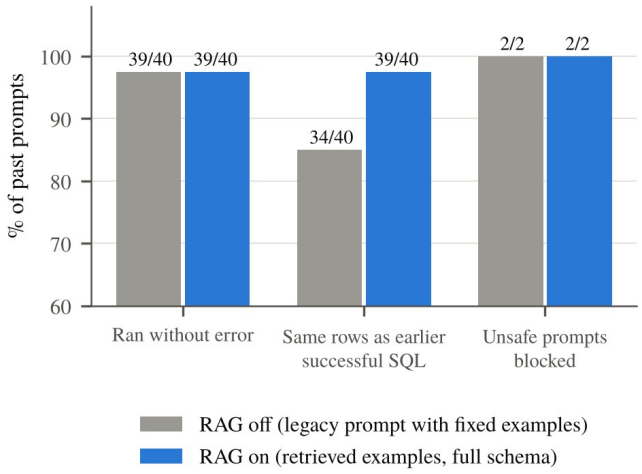


Fig. 7. Re-running 40 past user prompts and 2 unsafe prompts with retrieval off and on. Consistency counts prompts whose result matches the earlier successful result.

TABLE IV
RESULTS ON THE SPIDER DEVELOPMENT SUBSET (N = 32)

Config	EA % [95% CI]	Recall	Tables	Tokens
B: full schema	62.5 [45.3, 77.1]	1.00	8.56	611
C: + table RAG	62.5 [45.3, 77.1]	1.00	5.69	486
D: + example RAG	71.9 [54.6, 84.4]	1.00	8.56	920
E: both	68.8 [51.4, 82.0]	1.00	5.69	795

Recall = table recall@k after FK expansion (k = 4, at most 8 tables).

C. Effect of Table Retrieval

Table retrieval sent 4.8 of 11 tables on average and cut the prompt from 1,108 to 561 tokens, a 49% reduction. EA fell from 96.7% to 90.0%: two questions were lost and none gained ($p = 0.5$). On Spider, retrieval cut tables from 8.56 to 5.69 and tokens by 20% (611 to 486) with no change in EA and a table recall of 1.00.

To check recall on the project schema, we used the stored vectors of the 52 project questions as queries. Every question's reference tables were among the top four, giving recall@4 = 1.00, and expansion produced 4.9 tables on average, close to the 4.8 observed live. The two lost questions most likely come from ranking and distraction, not from missing tables. In "total salary expenditure per department", the university `departments` card ranks above `corp_departments` in the stored-vector ranking (Fig. 4), and the model summed instructor salaries instead of employee salaries. For "the highest paid employee in each department", the model found the right rows but left out the join that adds the department name. Both errors are identical to the errors the model made before annotations (B0), which the annotated full schema had fixed. A shorter, reordered context brought the same ambiguity back. Combining both kinds of retrieval (E) on Spider gave 68.8%, between C and D.

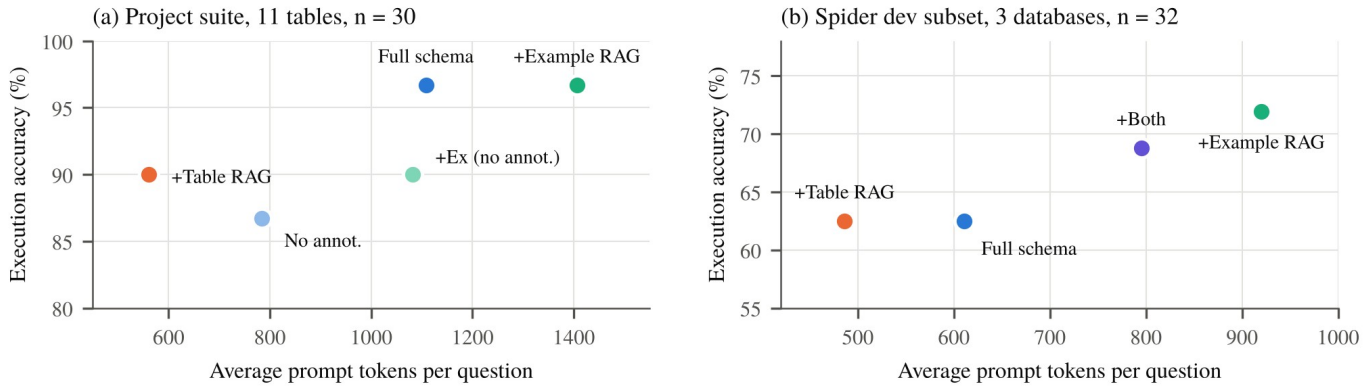


Fig. 8. Execution accuracy against mean prompt tokens. Each point is one configuration; the two panels use separate axes because the evaluation sets differ.

D. Accuracy, Cost and Latency

Fig. 8 puts the configurations on one accuracy-cost plane. On the project schema the best accuracy needs the full annotated schema (B, about 1,100 tokens); adding examples (D) costs about 300 more tokens for no gain, and table retrieval (C) halves the tokens for a small, non-significant loss. On Spider, examples give the highest accuracy, and table retrieval gives the same accuracy as the full schema at fewer tokens.

Retrieval adds one embedding call per question, which took about 0.6 s on average (605 ms for example retrieval and 644 ms for table retrieval, Fig. 9). Mean end-to-end latency ranged from 1.4 to 2.3 s across configurations, and generation time varied more than the retrieval overhead: configuration B averaged 1,819 ms in one run and 2,260 ms in another. Latency differences below about 0.5 s are therefore not meaningful in our setup.

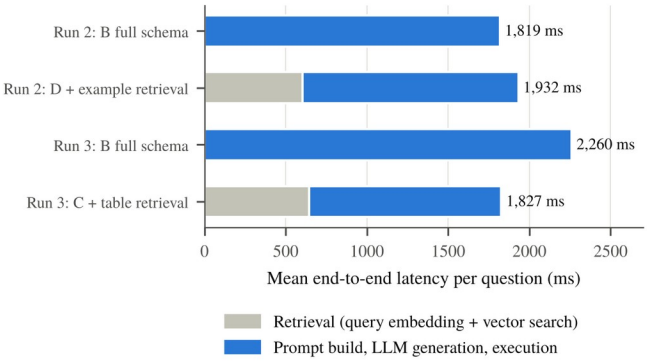


Fig. 9. Mean latency per question, split into retrieval time and the rest of the request, for the two paired runs.

E. Accuracy by Difficulty

Fig. 10 breaks EA down by difficulty. All configurations answered every easy question. Differences appear in medium and hard questions, which use joins, grouping and nested queries. Annotations raised hard-question accuracy from 78% to 100%, and table retrieval lowered it back to 78% through the two errors described above.

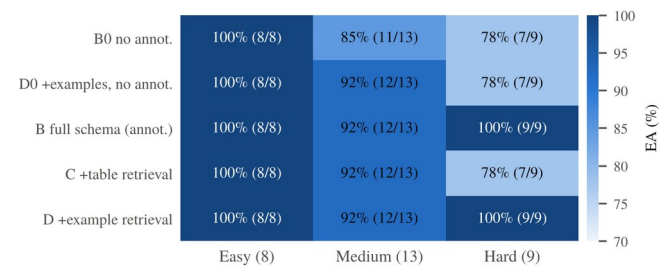


Fig. 10. Execution accuracy by question difficulty on the project suite (number of questions in brackets).

F. SQL Validator

The validator blocked all 25 attack queries, including stacked statements, a data-modifying common table expression, comment-hidden statements, `pg_sleep` time-based injection and mixed-case keywords. It blocked none of the 222 valid queries. Both unsafe prompts in the past-query set were also blocked in both modes. The validator works on keywords and statement structure. It does not stop read-only queries that expose data a user should not see, so a read-only database role remains a necessary second layer.

G. Discussion

Three lessons follow. First, on a small schema the cheapest accuracy gain is better schema text: descriptions and value hints helped more than any retrieval step. Second, example retrieval pays off when the schema or wording is unfamiliar, as on Spider and in real user prompts, and adds little once a small schema is well described. Third, table retrieval is mainly a cost tool. It halves the prompt with full recall, but similar tables from different domains can mislead the model, so it should be enabled by schema size or by a similarity margin rather than always. The leaked baseline A also shows why evaluation must check what the prompt contains: a perfect score can come from copying the answer key.

VIII. Limitations

The evaluation sets are small. With 30 questions, the 95% intervals span about 15 to 25 points, and none of the paired differences is significant at the 5% level. We used one generation model, and API latency varied between runs. The live runs logged how many tables were retrieved but not which ones, so our recall estimate for the project schema uses stored document vectors in place of live query vectors. The Spider subset covers three databases. Past-query consistency measures agreement with earlier answers, not correctness against a reference. We judge the prototype to be at Technology Readiness Level 4: validated in a laboratory setting, not yet with outside users.

IX. Conclusion and Future Work

We built and evaluated a Text-to-SQL system that combines PostgreSQL with a ChromaDB vector store to ground an LLM with retrieved tables and examples, and that validates every query before it runs. Schema annotations raised execution accuracy from 86.7% to 96.7%. Table retrieval cut prompt size by 49% with full table recall and a small loss on a small schema, and kept accuracy on Spider at 20% fewer tokens. Example retrieval raised Spider accuracy from 62.5% to 71.9% and agreement with earlier answers from 85.0% to 97.5%. The validator blocked every attack query without false blocks.

Future work will (i) log the retrieved table names and similarity scores to test whether a low top score predicts a wrong answer; (ii) switch table retrieval on only for large schemas or ambiguous matches; (iii) add an execute-and-repair loop that feeds database errors back to the model; (iv) evaluate on BIRD and larger samples with repeated runs; and (v) connect through a read-only database role.

Acknowledgment

We thank our course faculty for guidance during the project reviews.

References

[1] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. EMNLP*, 2018, pp. 3911-3921.

[2] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., “Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[3] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[4] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, “Text-to-SQL empowered by large language models: A benchmark evaluation,” *Proc. VLDB Endowment*, vol. 17, no. 5, pp. 1132-1145, 2024.

[5] R. Pedro, M. E. Coimbra, D. Castro, P. Carreira, and N. Santos, “Prompt-to-SQL injections in LLM-integrated web applications: Risks and defenses,” in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, 2025.

[6] M. Lin, H. Zhang, J. Lao, R. Li, Y. Zhou, C. Yang, Y. Cao, and M. Tang, “Are your LLM-based text-to-SQL models secure? Exploring SQL injection via backdoor attacks,” *Proc. ACM Management of Data*, vol. 3, no. 6, pp. 1-27, 2025.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459-9474.

[8] N. Rajkumar, R. Li, and D. Bahdanau, “Evaluating the text-to-SQL capabilities of large language models,” arXiv:2204.00498, 2022.

[9] X. Dong, C. Zhang, Y. Ge, Y. Mao, Y. Gao, L. Chen, J. Lin, and D. Lou, “C3: Zero-shot text-to-SQL with ChatGPT,” arXiv:2307.07306, 2023.

[10] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, and Z. Li, “MAC-SQL: A multi-agent collaborative framework for text-to-SQL,” in *Proc. COLING*, 2025, pp. 540-557.

[11] H. Li, J. Zhang, C. Li, and H. Chen, “RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13067-13075.

[12] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, “CHESS: Contextual harnessing for efficient SQL synthesis,” arXiv:2405.16755, 2024.

[13] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769-6781.

[14] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982-3992.

[15] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824-836, 2020.

[16] S. Xue, C. Jiang, W. Shi, F. Cheng, K. Chen, H. Yang, Z. Zhang, J. He, H. Zhang, G. Wei, et al., “DB-GPT: Empowering database interactions with private large language models,” arXiv:2312.17449, 2023.

[17] C. Xu, J. McAuley, and P. Wang, “Mirror: A natural language interface for data querying, summarization, and visualization,” arXiv:2303.08697, 2023.

[18] C. Zhang, Y. Mao, Y. Fan, Y. Mi, Y. Gao, L. Chen, D. Lou, and J. Lin, “FinSQL: Model-agnostic LLMs-based text-to-SQL framework for financial analysis,” in *Companion Proc. ACM SIGMOD Int. Conf. Management of Data*, 2024.

[19] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, “Next-generation database interfaces: A survey of LLM-based text-to-SQL,” arXiv:2406.08426, 2024.

[20] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, “A survey on employing large language models for text-to-SQL tasks,” *ACM Comput. Surv.*, 2025, doi: 10.1145/3737873.

[21] J. Lee, F. Chen, S. Dua, D. Cer, M. Shanbhogue, I. Naim, et al., “Gemini Embedding: Generalizable embeddings from Gemini,” arXiv:2503.07891, 2025.

[22] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209-212, 1927.

[23] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153-157, 1947.